

THE COMPLETE GUIDE

DevKit2023CustomLinux

From fresh Windows to KDE Plasma.
One launcher. One finished ISO.

Microsoft Windows Dev Kit 2023
Blackrock / Qualcomm SC8280XP
Debian 13 ARM64 + pinned patched kernel

1. PREPARE WINDOWS

2. BUILD YOUR ISO

3. WRITE USB

4. INSTALL LINUX

Important: the public project works with independent target preparation. Each finished ISO contains private firmware and identity for one selected Dev Kit. Publish the source, never the contents of ISO.

Owner-accepted 1.0.0 baseline. The working image is preserved unchanged. This guide does not certify every peripheral, another kit or an arbitrary Windows disk layout.

Contents

Part 1 / Beginner setup	3
1. Know what you are building	3
2. Get ready in Windows	3
3. Build with one launcher	3
4. Write the USB	4
5. Start the live desktop	4
6. Use the desktop	5
7. Install alongside Windows	5
8. Find a log or diagnose a problem	6
9. Clean builds and interrupted cleanup	6
10. Publish the project on GitHub	7
 Part 2 / Implementation reference	 8
1. Product and architecture	8
2. Source and patch pins	8
3. Board support and dependencies	9
4. Why prepared early boot matters	10
5. Firmware preparation and privacy	10
6. Bluetooth: the actual repair	11
7. Audio, search, Chrome and support tools	11
8. Installation implementation	11
9. Clean Windows build and repository organization	12
10. Sources and scope	12
 Part 3 / Sources and provenance	 13
Hardware and boot	13
Exact patch selection	13
Firmware and Bluetooth	14
Desktop and installer	14
Windows setup, USB and Chrome	14

Part 1 / Beginner setup

DevKit2023CustomLinux 1.0.0 - Microsoft Windows Dev Kit 2023.

This guide takes you from a normal Windows installation to a prepared USB, then to Linux alongside Windows. You do not need to type a series of commands. The same guide and technical reference are in the PDF.

1. Know what you are building

An **ISO** is a complete bootable disk image. **WSL** provides a Linux build environment inside Windows. The launcher builds Linux there, adds only your explicitly selected Dev Kit's required firmware and Bluetooth identity, checks the finished image, and removes temporary build files.

Your finished ISO belongs to that one kit. The source project is reusable on other Dev Kits; the prepared ISO is not. Never upload a prepared image to GitHub.

Already have the working ISO for this kit? You can go straight to section 4. You do not need to rebuild it or run an APT update to make the installer open.

2. Get ready in Windows

1. Use the kit's normal licensed **Windows 11 ARM64** installation. Finish its normal Windows and OEM driver updates. An unconfigured generic Windows image may not contain the necessary device firmware.
2. Turn on built-in Bluetooth once in Windows. This makes its controller address available to the exporter. Do not substitute an external Bluetooth dongle.
3. Make a separate backup of your important files. Keep Windows recovery media and any BitLocker recovery key somewhere other than this kit.
4. Download or copy the complete trusted project. Extract it into a writable local folder, not a ZIP, cloud-synced folder or shared drive. Keep all the source folders together.
5. Connect mains power and internet. Allow **several hours** and approximately **100 GB free** on the drive holding WSL as a planning allowance. Peak space varies; clean builds download and compile again every time.
6. Have a USB drive of at least **8 GB** that can be erased. Do not use your only Windows recovery drive or backup drive.

The supported builder is Windows ARM64 with native Debian 13 ARM64 WSL 2. An x64 Windows PC is not supported by this workflow. WSL uses its native Linux disk, not the Windows project folder, for compilation.

3. Build with one launcher

1. Double-click **Start-DevKit2023CustomLinux.cmd** in the project folder.
2. Choose **Build an ISO for THIS Dev Kit (recommended)**.
3. Confirm that this computer is the intended target. The dialog explains the firmware and identity being included; it does not collect your passwords, recovery keys, full registry or Bluetooth pairing keys.
4. If an ISO already exists, confirm replacement only if you want a new build. The existing ISO remains intact until the replacement passes verification. A successful replacement does not keep an old backup.

5. Approve administrator access when Windows asks about WSL setup. The launcher enables the needed Windows features and installs WSL/Debian when missing. It never deletes an existing Linux distribution.
6. If asked to restart Windows, close your other work, restart normally and open the **same launcher** again. Nothing schedules itself to run at login. An incompatible existing Debian is reported and left untouched.
7. Leave the console open while the source is built and the target firmware is authenticated. No cached base image or old target export is selected.
8. Wait for **Ready** and the USB handoff dialog. The only normal final output is **ISO/DevKit2023CustomLinux-1.0.0-arm64.iso** inside this project.

Build signatures, source consistency and image integrity checks stay enabled. These are automatic build checks, not a new request for manual hardware testing.

Temporary kernel sources, packages, root filesystems, manifests and the automatic target export are removed after success or an ordinary handled failure. WSL, Debian, installed compiler tools and normal operating-system records remain. A power cut, forced process kill or WSL failure can interrupt cleanup; no program can promise deletion after it has stopped running. See section 9.

Building on a different ARM64 computer

On the **target Dev Kit**, choose **Export THIS Dev Kit for a different ARM64 builder** and select a private destination. Transfer the resulting target-bundle folder privately. On the builder, choose **Build for another Dev Kit from its exported bundle** and select that exact folder.

This optional export is the only deliberate non-ISO output. Because it is your explicitly selected transfer file, the builder does not delete it. Remove your copy after the build if you no longer need it. A future rebuild can export again from the intended kit's Windows. Never commit the bundle to the source tree.

4. Write the USB

1. Open the project **ISO** folder. The launcher also has **Open ISO folder**.
2. Obtain the Windows **ARM64** version of [Rufus](#) from its official website. Rufus is not bundled or automatically executed.
3. Start Rufus and check its **Device** field carefully. Select only the USB you intend to erase, not the internal Windows disk or a backup drive.
4. Select the prepared ISO. If Rufus offers ISO mode versus DD mode, choose **DD/raw image mode** to preserve the boot layout.
5. Read and accept the USB erase warning. Wait for completion and safely eject it.

Copying the ISO as an ordinary file onto a formatted USB is not the same as writing a bootable USB. This project's launcher never formats or writes a drive.

5. Start the live desktop

1. Use the reference connections: **rear USB-A** for the USB and a keyboard/mouse, and **mini-DisplayPort** for the monitor.
2. Shut the kit down. With the USB connected, use Power + the UEFI button to enter firmware setup. Follow Microsoft's [Dev Kit UEFI guide](#).

3. This image is unsigned. **Secure Boot must be disabled** for it. Have Windows recovery access available before changing boot/security settings.
4. In Boot Configuration, choose USB Storage.
5. Select **Install DevKit2023CustomLinux** in GRUB. This starts the live desktop; it does not immediately install or change the internal disk.

The required **clk_ignore_unused pd_ignore_unused** parameters are already included. Do not remove them. Do not add **efi=noruntime**: installation needs EFI runtime services to create its Linux boot entry.

Firmware is already present in the prepared image. Linux does not try to scan or unlock Windows for firmware during boot. Preparing an image from running Windows does not require decrypting C:. This does not remove normal BitLocker recovery risks when boot settings change.

6. Use the desktop

- **Install DevKit2023CustomLinux** opens the guided Windows-alongside installer.
- **Install Google Chrome** asks for confirmation and installs the current official Linux ARM64 stable package over the internet. In a live session, that installation is temporary; install it again after installing Linux to keep it.
- **Diagnostics** creates an optional report when you request one, shows it and offers a readable file you can copy. Review it before sharing privately.
- **Konsole** is available for commands. Dolphin is the file manager.

Wi-Fi and Bluetooth are controlled in KDE's system tray. Select the correct audio output in Sound settings. The image includes the backend required for KDE's speaker-test buttons as well as normal PipeWire playback.

Windows and Linux use the controller's same factory address but do not share pairing keys. An accessory with an old/conflicting bond may need to be re-paired. Sharing or copying pairing secrets is not automated.

The live desktop is non-persistent. Save files or diagnostic reports to a separate writable drive before restarting; session changes disappear on reboot.

7. Install alongside Windows

Prepare free space in Windows

1. Back up your data and keep recovery access available. There is no promise that changing firmware settings can never trigger Windows recovery.
2. Open **Disk Management** in Windows.
3. Shrink the Windows volume to leave **64 GiB unallocated** if possible (**32 GiB minimum**). Leave it unallocated: do not create or format a partition.
4. Keep all existing Windows, EFI, Microsoft Reserved and Recovery partitions.
5. Disable Fast Startup/hibernation for this shutdown or use a full Windows shutdown. Do not boot Linux against a hibernated Windows session.

If Windows cannot shrink safely, stop. The Linux installer intentionally does not resize or decrypt Windows. Do not remove recovery partitions to make it proceed.

Run the desktop installer

1. Boot your prepared USB and open **Install DevKit2023CustomLinux**.
2. Approve the administrator prompt. The first checks are read-only.
3. Read the proposed internal disk, Linux size and EFI reuse. It must propose **one new Linux partition in unallocated space**, not a Windows partition.
4. Continue to the installer. Choose language, keyboard and time zone, then create your own account and password.
5. Read the final summary. Only the final **Install** confirmation starts writing. Remain on mains power and do not remove the USB during installation.
6. After completion, restart and remove the USB. GRUB offers **DevKit2023CustomLinux** and **Windows Boot Manager**.
7. Use the normal system from the internal disk. You no longer need the USB. If you want Chrome permanently, use its installer in installed Linux.

The supported layout is one internal NVMe Windows GPT disk with one existing ARM64 Windows EFI partition, at least **32 MiB free in EFI**, no previous Linux installation, and an adequate unallocated gap. Mounted, ambiguous or changed layouts are refused. Close file-manager windows accessing the internal disk.

The installer rechecks the plan before writing, formats only its new ext4 Linux partition, preserves original EFI files and existing boot entries, and adds Linux ahead of the preserved boot order. It does not support reinstall/overwrite, erase, replacement or automatic rollback after a partial write.

If installation fails after writing begins, **do not repeat it blindly**. Save the failure information and request review. Do not delete partitions or EFI entries to bypass a refusal. Windows Boot Manager should remain available in the kit's UEFI; keep recovery media available for unexpected failures.

8. Find a log or diagnose a problem

No manual APT update is needed to launch the complete image's installer. Updating packages is not a substitute for understanding a failure.

If an installer warning closes without the main window, the error dialog offers **Open log**. You can also open Dolphin, press **Ctrl+L**, enter `~/local/state/devkit2023customlinux/` and open the latest installer log. That is the address bar, not the search field. This folder is hidden because its name begins with a dot; Ctrl+H shows hidden files.

Run **Diagnostics** before restarting if you want a support report. Copy it to writable storage, not into the read-only ISO filesystem. Private startup logs may contain details beyond the redacted report: inspect them before sharing.

For no signal, first confirm the ISO belongs to this kit, use miniDP and rear USB-A, and retain the default clock/power parameters. Capture the last error if possible. Do not invent firmware files or re-enable retired boot-time imports.

9. Clean builds and interrupted cleanup

A completed normal build leaves only the ISO in **ISO/**. It does not leave the old sibling Data folder, automatic logs, source archives or previous builds.

Temporary work is restricted to newly created directories named:

- Windows temporary storage: **DevKit2023CustomLinux-build-** plus a random ID.
- Native WSL storage: **/var/tmp/DevKit2023CustomLinux-build.** plus a random ID.

Ordinary success/failure handlers remove the invocation's files. Never force-close the console while compiling. If power loss or a forced shutdown interrupts cleanup, restart Windows and use **Remove interrupted build files** in the launcher once no build is running. It removes only owned, marked temporary build directories and partial publication files, never WSL itself or an external target bundle. Any refused/busy path is left intact and reported.

WSL and its installed build tools are prerequisites, not old ISO build leftovers. Deletion inside WSL frees Linux filesystem space; Windows may not immediately shrink WSL's virtual-disk file. This workflow does not resize that virtual disk.

10. Publish the project on GitHub

Publish the **source folder**, including this guide, the PDF, scripts, patches, configuration and tests. Keep its structure; these folders are the actual product.

Exclude ISO/ and any independently exported private bundle. Git ignores ISO automatically, but a browser/manual upload can still expose it if you select it. Do not use force-add on ignored images.

No GitHub repository or upload is created automatically. The source can be placed in a new empty repository, using GitHub Desktop or Git. Project code is MIT; upstream licenses remain attached. A private prepared ISO is not a distributable universal image.

The technical details and pinned patch list are in IMPLEMENTATION.md. Current scope and limitations are in VALIDATION.md.

Part 2 / Implementation reference

DevKit2023CustomLinux 1.0.0. This describes the implemented source, not a promise that every capability of the hardware has been validated. The owner accepted the working edition; the runtime and pinned kernel are unchanged by repository cleanup.

1. Product and architecture

The target is Microsoft Windows Dev Kit 2023, board name Blackrock, using Qualcomm SC8280XP (Snapdragon 8cx Gen 3 platform). The kernel boots ARM64 in EL1 with the Blackrock device tree. This is not an x86 image and not a generic ARM computer ISO.

Userspace is Debian 13 Trixie ARM64 created by debootstrap minbase. Essential KDE Plasma, KWin Wayland, SDDM, Dolphin, Konsole, NetworkManager, BlueZ, PipeWire, WirePlumber and Calamares are installed without recommended optional packages. Dependencies still bring many supporting libraries; minimal does not mean a desktop can run without its normal services and libraries.

Debian repositories are Trixie, updates, security and backports. Qualcomm, Atheros and Realtek firmware packages come from backports. Package versions are not snapshot-frozen: a fresh build may contain newer Debian security updates. The kernel, patch files, topology and security-sensitive verifier are pinned. This is a source-controlled build recipe, not a byte-for-byte reproducible Debian snapshot or a supported rolling kernel-update service.

2. Source and patch pins

The authoritative values live in scripts/lib/common.sh and are checked during building. Three patch mailbox files apply four commits.

Input	Pin or purpose
Kernel source	linux-next, tag next-20260902
Kernel commit	32b6ef9a5d0eca44f9cd91f52f4faa89f145a0de
Kernel release	7.3.0-rc1-next-20260902-devkit2023customlinux
Device tree	sc8280xp-microsoft-blackrock.dtb
20260825-blackrock-dp-audio-altmode-v2.mbox	Two commits: replace obsolete sound nodes with DP audio; enable four DP lanes on the board endpoints
20250609-blackrock-usb0-mux-v1.mbox	Correct USB0 SBU-mux enable GPIO polarity for connector orientation
20260902-buffer-fix-null-b-folio.mbox	Guard a NULL buffer folio during journal write submission
AudioReach topology	jglathe/audioreach-topology, commit 31e0451c67b9001b49118177617a3e8b7bbd4c5c
ALSA UCM routes	jglathe/alsa-ucm-conf, commit 77b4b779ab42cc37ddd18052732aa4dc1e3fb37e

Input	Pin or purpose
Catalog verifier	osslsigcode 2.14, built from its hash-checked upstream archive

The complete patch SHA-256 values are:

- DP/audio v2: 6c9d6cab985494ee4e90142c74ab6e85f62df4da59f17059feeb9b6ff55042bf
- USB0 mux: 380a6b885e23c607955e67b7c8cb36baff0b3c7dad0b099049f0899e94b43300
- Buffer fix: d469ac0df3a69765f3c5b1a1d45aaa5dded6c512f44a64ac332ca7f8e6a26ad9

These are a reviewed selection, not every LKML patch. The supplied August 23 audio link describes an earlier series; the checked-in **August 25 v2** is what this project applies. Do not stack older versions of the same patch.

The buffer fix addresses the NULL-folio journal-write failure found during the earlier isolated VM investigation. It is not claimed as an explanation of a mini-DisplayPort signal-loss event.

3. Board support and dependencies

Function	Implemented Linux path	Important dependency
Display/GPU	DRM MSM, SC8280XP display clocks, Blackrock DTB	Authenticated target graphics firmware, early loading
DP audio	SC8280XP ASoC, QDSP6/AudioReach, ALSA UCM	Target ADSP firmware, board topology, DP sink
Wi-Fi	ath11k PCI, cfg80211/mac80211	Packaged Atheros/Qualcomm firmware
Bluetooth	HCI UART QCA, btqca, BlueZ/Bluedevil	WCN6855 NVM selection and the target's public address
Ethernet	USB RTL8152	Built-in Ethernet device and normal USB stack
USB	DWC3 Qualcomm, XHCI, Type-C/PMIC GLINK	Board mux GPIO, power, PHY and interconnect definitions
NVMe	Qualcomm PCIe, NVMe	Board regulators/clocks and GPT layout
Boot/storage	EFI, ext4, FAT, SquashFS/Zstd, OverlayFS	ARM64 EFI loader and matching initramfs

config/kernel/blackrock.config enables the relevant Qualcomm pinctrl, PMIC, regulators, remote processors, DMA, interconnects, clocks, PHYs and storage. The kernel builder verifies the WCN6855 Bluetooth compatible, TLMM GPIO 133 Bluetooth enable, TLMM GPIO 100 active-low USB0 mux control, sound model, four-lane DP endpoints, GPI DMA masks and board-ID-aware Bluetooth NVM selection.

The authoritative pin and endpoint mapping is the pinned kernel's Blackrock DTS plus the checked-in patches. No guessed board wiring or manually invented Bluetooth address is compiled into the DTB. Linux uses Linux drivers; the exported Windows files are firmware, not Windows .sys driver execution.

4. Why prepared early boot matters

The boot sequence is deliberately ordered:

1. ARM64 UEFI loads GRUB and the Blackrock DTB.
2. GRUB supplies `clk_ignore_unused` and `pd_ignore_unused`. These retain clocks and power domains otherwise considered unused while this platform starts.
3. Automatic loading of `msm`, `qcom_q6v5_pas` and `snd_soc_sc8280xp` is blacklisted. This prevents those drivers starting before the prepared firmware checks.
4. The `initramfs` hook checks Blackrock compatibility, the hashed SMBIOS binding and firmware integrity before normal live-storage discovery.
5. After successful checks it explicitly loads those three modules. It clears inherited `MODPROBE_OPTIONS` for these deliberate loads and checks that each module actually appeared in `sysfs`. A successful command alone is insufficient.
6. System services revalidate the prepared data and sequence display, audio, Bluetooth address provisioning and the desktop.

The deliberate module loading does not bypass firmware authentication. The public base lacks a target binding and therefore stops before hardware startup. A wrong-target image stops too; it does not proceed with substitutes.

The topology is included at the kernel-requested location:

`usr/lib/firmware/qcom/sc8280xp/SC8280XP-MICROSOFT-BLACKROCK-tpkg.bin`, both in the root filesystem and early image. Firmware copied only after the desktop starts would be too late for this workflow.

`efi=noruntime` is not a default. It disables EFI runtime access needed by the installer's boot-entry step. It is not a universal display fix. Secure Boot is unsupported because the custom boot chain is unsigned.

5. Firmware preparation and privacy

The explicit Windows exporter collects exactly three firmware payloads: `qcadsp8280.mbn`, `qccdsp8280.mbn` and `qcdxkmsuc8280.mbn`, with their original INF and CAT evidence. It does not copy the full DriverStore, Windows drivers, user files, registry hives, recovery keys, credentials or accessory pairing secrets.

The preparer independently verifies Microsoft catalog signatures and exact SHA-256 membership of both INF and MBN. Digestless catalog entries do not qualify. The highest authenticated DriverVer wins; conflicting same-version material is rejected. Only the three selected packages, nine evidence/payload files, are embedded and reverified. Another board's Lenovo SC8280XP firmware directory is removed from the generic base so it cannot substitute for Blackrock firmware.

The export holds a SHA-256 binding to the target's canonical SMBIOS UUID and its built-in Qualcomm Bluetooth factory public address. Missing or ambiguous identity fails; no disk/MAC/other-machine identity is substituted. A stable hash is still an identifier, not anonymity or hardware attestation.

The prepared ISO contains this private target material. The public source and temporary generic base do not. The collection/preparation protocol is `prepared-workflow-v1`. See `PREPARATION.md` for its exact binding format.

6. Bluetooth: the actual repair

The pinned kernel already has the QCA UART driver and board-ID-based WCN6855 NVM selection. This project does not add an arbitrary old Bluetooth patch on top.

At boot the helper finds exactly one built-in WCN6855 controller through its device-tree ancestry, rather than assuming every hci0 is the right device. It unblocks that controller only, verifies the management reply, and provisions the prepared target's address only when the controller is positively identified as unconfigured and missing its public address.

A controller already configured with another address is rejected. Pairing keys are not imported or synthesized. BlueZ is ordered after address provisioning.

The userspace service repair supplies an empty input **pipe** to btmgmt. The BlueZ 5.82 shell path used here registers stdin with epoll; systemd's /dev/null can prevent the command from reaching the controller. The helper uses bounded timeouts, validates replies even after exit status zero, and handles the global unconfigured-controller list correctly. This is distinct from a missing kernel driver or replacing firmware.

A connected Bluetooth speaker was observed. That does not establish every phone profile, accessory, range, reconnect or simultaneous-radio case.

7. Audio, search, Chrome and support tools

The DP audio patches, target ADSP firmware, AudioReach topology and Blackrock UCM routes provide the board's audio path. PipeWire/WirePlumber expose it to KDE. The KDE speaker-test popup saying No such driver was a missing libcanberra Pulse backend: libcanberra-pulse is now explicitly installed. It was not proof that Bluetooth's kernel driver was missing.

Dolphin's Invalid protocol filenamesearch error was a missing KIO worker. kio-extras is explicitly included, and image verification checks its worker file. No manual APT update is required for these delivered components.

Chrome is optional. The desktop installer downloads Google's signing key, accepts only the pinned primary fingerprint EB4C1BFD4F042F6DDDCCEC917721F63BD38B4796 and its subkeys, scopes it to the official ARM64 APT repository, and installs current stable google-chrome-stable:arm64. The binary and repository key are not prebundled. Upstream key rotation may require a reviewed installer update; signature failures are not bypassed.

Diagnostics 1.4 is a permanent, optional product support tool, not a directory of development checkup files. Its hidden implementation presents a KDE progress dialog, redacts selected identifiers and offers a text report/viewer. No report exists in a fresh image until the user requests one. Tests and project documents are not copied into the live overlay.

8. Installation implementation

The single windows-free-space-v1 policy uses existing unallocated space on one internal Windows GPT NVMe disk. It has no erase/replace/resize paths. Read-only preflight checks the target, readiness, EFI runtime, Secure Boot state, existing partitions, EFI free space and absence of ambiguous/mounted layouts.

The normal-user launcher receives a readable plan summary over the authorized preflight's stdout. Root-owned JSON and Calamares summary cache remain mode 0600. Private-log umask is reset to 022 before privileged children run. Errors retain the real exit status and offer Open log. Calamares uses the packaged executable with its existing policy authorization, XCB and restricted temporary root display access. The branding slideshow is a supported image list.

Before writing, the backend rechecks disk identity, complete GPT, original EFI file hashes and boot variables. It appends only the new Linux partition, disables signature wiping for existing devices, formats only that new ext4 partition and verifies Calamares's mount targets.

Target firmware/address records and early boot data are transferred into installed Linux. No live pairing keys are copied. The user's account is created during setup. GRUB is installed under EFI/DevKit2023CustomLinux without overwriting Microsoft or fallback EFI files. A Linux boot entry is added while original entries and relative ordering are preserved. Linux and Windows get explicit GRUB entries.

A partial write is not automatically rolled back or blindly retried. Recovery media and backups remain necessary. Full safeguards and maintainer entry points are in INSTALLATION.md.

9. Clean Windows build and repository organization

The supported launcher always makes a fresh native Linux workspace. Source is copied from the explicit project inventory, never by copying arbitrary root contents. Temporary public-base/manifests/checksums are used internally for verification; the selected target is then prepared in a private mount namespace.

Windows checks the preparation completion metadata and copied ISO bytes before atomically publishing only ISO/DevKit2023CustomLinux-1.0.0-arm64.iso. Replacing an existing image requires confirmation; it is not removed before the new image is ready. The final source/ISO folders contain no automatic build logs or history.

Normal completion and handled failures clean the run's native and Windows temporary directories. An interrupted-build cleanup action recognizes private ownership records, refuses active builds and linked/unexpected paths, and never deletes external bundles or the installed WSL distribution. Power loss can interrupt cleanup; prerequisites and normal OS metadata are not build outputs.

The repository's explicit file map rejects unmapped content, stale indexes, missing service executables and private payload extensions. ISO is intentionally excluded from source and Git. Only the named public guide PDF is allowed as a document asset. Synthetic tests remain maintenance source, not old build history.

This cleanup preserves the accepted ISO byte-for-byte. It does not rebuild, upgrade or change the running board support. New build orchestration was reviewed with static checks only, at the owner's request; a fresh full build was not repeated for this handoff.

10. Sources and scope

Primary source links and version context are in UPSTREAM.md. The checked-in patch files retain their upstream authorship and notices. MIT covers project-authored code, not all software inside an assembled ISO.

There is no bundled custom Windows, no Chrome binary, no public private-firmware release, no blanket all-drivers certification and no promise of unattended installation on every Windows partition layout. Suspend/resume, NPU, hardware video codecs, EL2/KVM and Secure Boot are outside the claimed release scope.

Part 3 / Sources and provenance

Reference date: 6 September 2026. The implementation reference records exactly what this project uses; a linked repository is not automatically imported in full. Read the checked-in pins before adopting a newer upstream change.

Hardware and boot

- **Microsoft Windows Dev Kit 2023 guide**
<https://learn.microsoft.com/en-us/windows/arm/dev-kit/>: target platform, miniDP pre-boot display, UEFI and USB boot.
- **Jens Glathe's Linux Dev Kit repository**
https://github.com/jglathe/linux_ms_dev_kit: Blackrock platform development and wider Qualcomm context. This project builds its own pinned linux-next selection, not that repository's current moving branch.
- **Linux Surface organization**
<https://github.com/linux-surface>: related community context, not a blanket dependency or source of every driver in this image.
- **Kernel parameter reference**
<https://www.kernel.org/doc/html/latest/admin-guide/kernel-parameters.html>: clock/power-domain controls and EFI runtime semantics.
- **Pinned linux-next source**
<https://kernel.googlesource.com/pub/scm/linux/kernel/git/next/linux-next.git/+32b6ef9a5d0eca44f9cd91f52f4faa89f145a0de>: base for the source-built kernel.

Exact patch selection

- **Original DP audio and four-lane discussion**
<https://lkml.iu.edu/2608.2/12170.html>: August 23 cover letter supplied during research. This is background; the project applies the August 25 v2 mailbox.
- **DP audio v2**
<https://lore.kernel.org/all/20260825-blackrock-audio-v2-1-659cc4724d57@oldschoolsolutions.biz/>: Jens Glathe, replace obsolete sound nodes and enable DP audio; the local mailbox also contains the second four-lane altmode commit.
- **USB0 mux correction**
<https://lore.kernel.org/all/20250609-blackrock-usb0-mux-v1-1-7903c3b071e4@oldschoolsolutions.biz/>: Jens Glathe, USB0 enable GPIO polarity. Canonical message IDs are retained in the checked-in mailbox even when an archive is unavailable.
- **NULL buffer-folio fix**
<https://lkml.iu.edu/2609.0/02958.html>: Joseph Qi, guard the folio check in journal write submission.
- **AudioReach topology pin**
<https://github.com/jglathe/audioreach-topology/tree/31e0451c67b9001b49118177617a3e8b7bbd4c5c>: Blackrock topology used by the build.
- **ALSA UCM pin**
<https://github.com/jglathe/alsa-ucm-conf/tree/77b4b779ab42cc37ddd18052732aa4dc1e3fb37e>: board audio routes; the source notice and BSD-3-Clause license are retained under the overlay.

The local patch files and hashes in scripts/lib/common.sh are authoritative for this edition. The two lore message pages were not retrievable by this session's browser; their provenance comes from the retained original mailbox headers. No extra legacy Bluetooth patch is applied: the pinned kernel already contains the required board-ID firmware selection checked by the build.

Firmware and Bluetooth

- **Windows-side Blackrock firmware collection**
https://github.com/jglathe/wdk2023_fw_fetch: device-community reference for Windows firmware acquisition.
- **Microsoft SMBIOS UUID documentation**
<https://learn.microsoft.com/en-us/windows/win32/cimwin32prov/win32-computersystemproduct>: identity property used for target binding.
- **osslsigncode 2.14**
<https://github.com/mtrojnar/osslsigncode/tree/2.14>: pinned catalog verification implementation.
- **BlueZ 5.82 shell**
<https://github.com/bluez/bluez/blob/5.82/src/shared/shell.c>: stdin/event-loop behavior relevant to the service command pipe.
- **BlueZ management API**
<https://github.com/bluez/bluez/blob/5.82/doc/mgmt-api.txt>: controller configuration/public address operations.

Our helper's input-pipe correction and explicit reply validation are project userspace integration, not a claim that the kernel Bluetooth driver is replaced.

Desktop and installer

- **KDE 6.3.4 speaker test**
<https://github.com/KDE/plasma-pa/blob/v6.3.4/src/speakertest.cpp>: the libcanberra Pulse backend used by the test buttons.
- **Debian libcanberra-pulse**
<https://packages.debian.org/trixie/libcanberra-pulse>: explicit package needed for that backend.
- **Debian kio-extras**
<https://packages.debian.org/trixie/kio-extras>: filename-search worker and related KIO components.
- **Calamares 3.3.14 branding**
<https://github.com/calamares/calamares/blob/v3.3.14/src/branding/default/branding.desc>: slideshow/image-list contract.
- **Calamares mount job**
<https://github.com/calamares/calamares/blob/v3.3.14/src/modules/mount/main.py> and [fstab job](https://github.com/calamares/calamares/blob/v3.3.14/src/modules/fstab/main.py)
<https://github.com/calamares/calamares/blob/v3.3.14/src/modules/fstab/main.py>: integration used after the project's guarded partition plan.
- **sfdisk reference**
<https://man7.org/linux/man-pages/man8/sfdisk.8.html>: GPT operations used by the narrow storage backend.

Windows setup, USB and Chrome

- **Microsoft WSL installation**
<https://learn.microsoft.com/en-us/windows/wsl/install> and [command reference](#)
<https://learn.microsoft.com/en-us/windows/wsl/basic-commands>: setup, distribution installation and WSL 2.

- [Rufus official downloads](#)

<https://rufus.ie/en/>: choose Windows ARM64 and check the USB target before writing.

- [Google Chrome installation requirements](#)

<https://support.google.com/chrome/answer/95346?hl=en-GB>: Linux ARM64 support.

- [Google Linux repositories](#)

<https://www.google.com/linuxrepositories/>: official package/key distribution.

Chrome remains an optional current-stable download. Its earlier successful isolated package installation is not a GPU/browser hardware test. The current working ISO is retained; this documentation does not mean the repositories or drivers were upgraded during cleanup.